

Programación Avanzada

Gustavo Márquez Flores

¿ Que es un programa correcto ?

¿ Que esté bien escrito?

¿ Que sea óptimo? Memoria ? Tiempo ?

¿ Uso eficiente del lenguaje ?

¿ Algoritmo eficiente ?

+ Definición Básicas.

- ✓ Lenguajes de programación.
- ✓ Algoritmo.
- ✓ Programa.

+ Técnicas de programación y solución de problemas.

- ✓ Divide y vencerás.

Funcion Divide_y_Vencerás(P : Problema): Solución

{ Devuelve la solución del problema P }

Inicio

Si P es suficientemente pequeño o simple

Entonces devolver(solucion_simple(P))

Si_No

descomponer P en k partes (P1,P2,...,Pk);

Para (i=1; i<=k ;i++)

Si = Divide_y_Vencerás(Pi);

devolver(combinar(P1, P2, ..., Pk));

Fin

✓ Backtrack.

Una solución de un problema.

Todas las soluciones de un problema.

Mejor solución de un problema.

```
Funcion Backtracking (Etapa i solucionesCompletas) devuelve: boolean
Inicio
    Éxito = falso;
    IniciarOpciones(i, GrupoOpciones o);
    Repetir
        SeleccionarnuevaOpcion(o, Opcion n);
        Si (Aceptable(n)) entonces
            AnotarOpcion(i, n);

            SiSolucionCompleta(i) entonces
                Éxito = verdadero;
                ApuntarSoluciónCompleta(i, solucionesCompletas)

            Sino
                Éxito = Backtracking(i+1);
                Si Éxito = false entonces
                    cancelamosAnotacion(i, n);
                finsi;
            Finsi;
        Finsi;
    Hasta (NoQuedanOpciones(o));
    Retorna Éxito;
Fin;
```

✓ Ejemplos:

Multiplicar dos números muy grandes.

Búsqueda Binaria.

Cuadros mágicos.

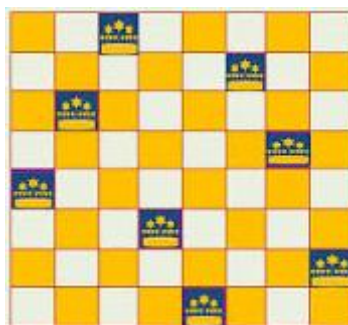
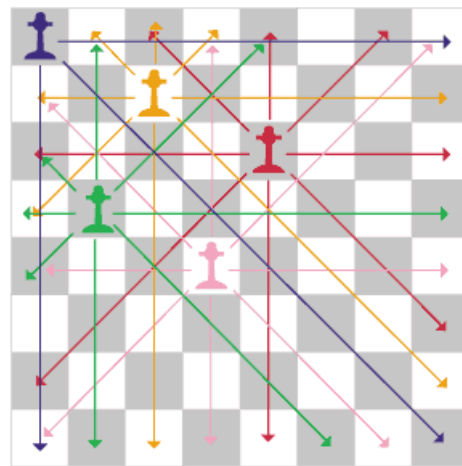
Cuadro mágico de 3 x 3 con suma 15

4	9	2
3	5	7
8	1	6

Cuadro mágico de 5 x 5 con suma 65

<i>17</i>	<i>24</i>	<i>1</i>	<i>8</i>	<i>15</i>
<i>23</i>	<i>5</i>	<i>7</i>	<i>14</i>	<i>16</i>
<i>4</i>	<i>6</i>	<i>13</i>	<i>20</i>	<i>22</i>
<i>10</i>	<i>12</i>	<i>19</i>	<i>21</i>	<i>3</i>
<i>11</i>	<i>18</i>	<i>25</i>	<i>2</i>	<i>9</i>

- ✓ Colocar 8 reinas en un tablero de ajedrez sin que se eliminen



Métodos para la verificación de problemas.

- ✓ Demostrar que el siguiente código es correcto:

$\{ \} \ c := a + b; c := c / 2 \ \{ c = (a + b) / 2 \}$

- ✓ Demostrar que es correcto :

$\{ \} \text{ if } a > b \text{ then } m := a \text{ else } m := b \ \{ (m \geq a) \wedge (m \geq b) \}$

- ✓ Encontrar el invariante asociado al siguiente código:

```
sum:=0;
j:=0;
while j<>n do begin
    sum:=sum+a;
    j:=j+1
end:
```

- ✓ Considerar la siguiente subrutina en pseudocódigo:

```
Subroutine ? ( N, M; R )
  R ← 1
  While ( M > 0 )
    R ← R x N
    M ← M – 1
  Return
End Subroutine
```

¿ Que realiza la subrutina ?

- ✓ Considerar la siguiente función en pseudocódigo:

```
Function Cuadrado( A )
  C ← 0
  D ← 0
  While ( D ≠ A )
    C ← C + A
    D ← D + 1
  Return C
End Function
```

Demostrar que esta función calcula el cuadrado de un entero positivo A.

- ✓ Pruebas por inducción.

Función 91 de John McCarthy.

La función está definida de la siguiente manera:

$$M(n) = \begin{cases} n - 10, & \text{si } n > 100 \\ M(M(n + 11)), & \text{si } n \leq 100 \end{cases}$$

Demostrar que la función obtiene $M(n) = 91 \forall$ entero $n \leq 100$ y $M(n) = n - 10$ para $x > 100$.

```
public static int mc91( int n )
{
    if (n > 100)
        return n - 10;
    else
        return mc91( mc91( n + 11 ) );
}
```

Paradigmas de programación.

- ✓ ¿ Que es un paradigma de programación ?
- ✓ ¿ Que paradigmas de programación existen y cuáles son las ventajas de cada uno de ellos ?
- ✓ El paradigma de programación lógico.
 - El lenguaje de programación lógico PROLOG.
 - Deducción lógica.
 - Ejemplo 1.

- Todos los que ayudan a Juan viven en casa de Manolo.

- Antonio ayuda a todos los que trabajan con él.

- Juan trabaja con todos los amigos de Carlos.

- Antonio es amigo de Carlos.

- ¿ Antonio vive en casa de Manolo ?

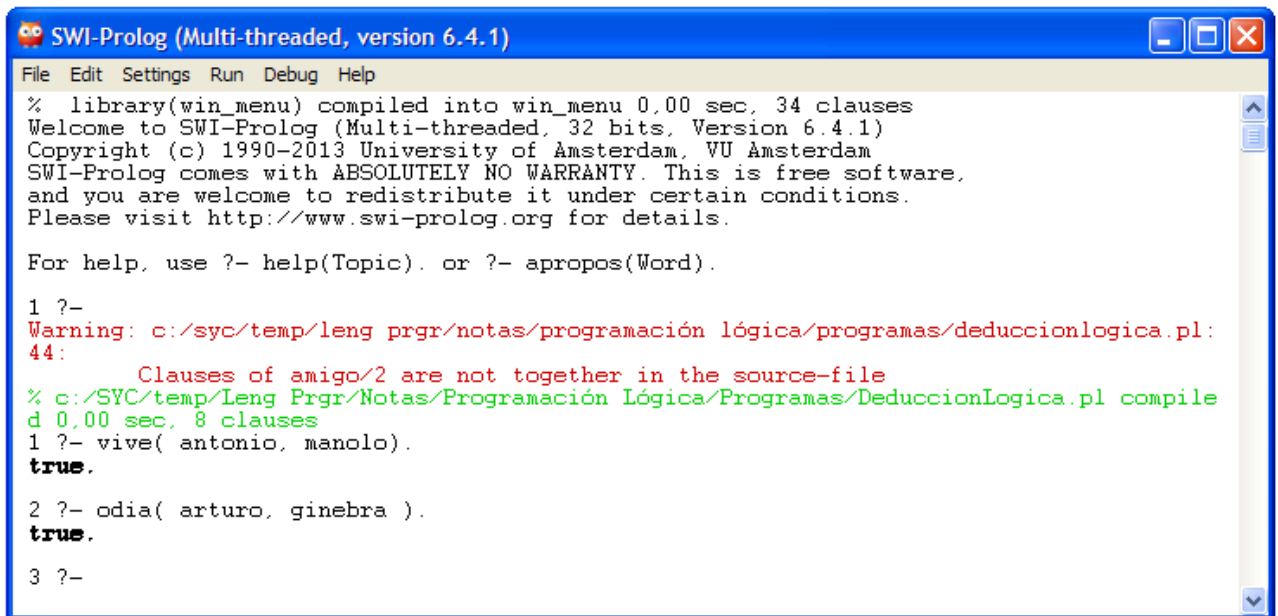
Meta : vive(antonio, manolo)

```
vive( X, manolo ) :- ayuda( X, juan ).
amigo( Z, carlos ) :- trabaja( juan, Z ).
amigo( antonio, carlos ).
```

▪ Ejemplo 2.

- Lantarote ama a la reina Ginebra.
- Lantarote no ama a ninguno de sus amigos.
- El rey Arturo es amigo de Lantarote.
- Los amigos de Lantarote odian a aquellos a quienes Lantarote ama.
- ¿ Arturo odia a la reina Ginebra ?

```
Meta: odia( arturo, ginebra ).
amigo( lanzarote, ginebra ).
ama ( arturo, lanzarote ).
amigo( W, lanzarote ) :- odia( W, Y ), ama( lanzarote, Y ).
```



```
SWI-Prolog (Multi-threaded, version 6.4.1)
File Edit Settings Run Debug Help
% library(win_menu) compiled into win_menu 0,00 sec, 34 clauses
Welcome to SWI-Prolog (Multi-threaded, 32 bits, Version 6.4.1)
Copyright (c) 1990-2013 University of Amsterdam, VU Amsterdam
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software,
and you are welcome to redistribute it under certain conditions.
Please visit http://www.swi-prolog.org for details.

For help, use ?- help(Topic). or ?- apropos(Word).

1 ?-
Warning: c:/syc/temp/leng prgr/notas/programación lógica/programas/deduccionlogica.pl:
44: Clauses of amigo/2 are not together in the source-file
% c:/SYC/temp/Leng Prgr/Notas/Programación Lógica/Programas/DeduccionLogica.pl compile
d 0,00 sec, 8 clauses
1 ?- vive( antonio, manolo ).
true.
2 ?- odia( arturo, ginebra ).
true.
3 ?-
```

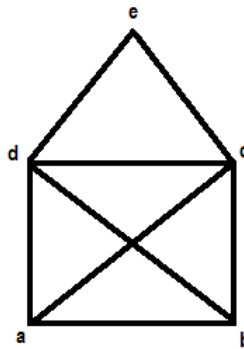
▪ Representación y procesamiento simbólico.

$$f(x) = (x^2 + x)(x - 3x^4)$$

$$\Rightarrow f'(x) = (2x + 1)(x - 3x^4) + (x^2 + x)(1 - 3 \cdot 4x^3)$$

▪ Representación de conocimiento e inferencia.

Dibujar la siguiente figura de un sólo trazo sin levantar el lápiz :



```
dibuja( Aristas, [ V1, V2 | Z ] ) :- eligevert( Aristas, Z, AristasM ),
                                     dibuja( AristasM, [ V2 | Z ] ).
```

```
eligevert( [ V1-X | RV ], V1-Z, RV ).
```

```
eligevert( [ V1 | Y ], V2, [ V1 | W ] ) :- eligevert( RV, V3, X ).
```

% Meta:

```
% dibuja( [a-b,a-c,a-d,b-c,b-d,c-d,c-e,d-e], L ).
```

```
% dibuja( [d-e,e-c,d-c,d-a,a-c,d-b,c-b,a-b], L ).
```

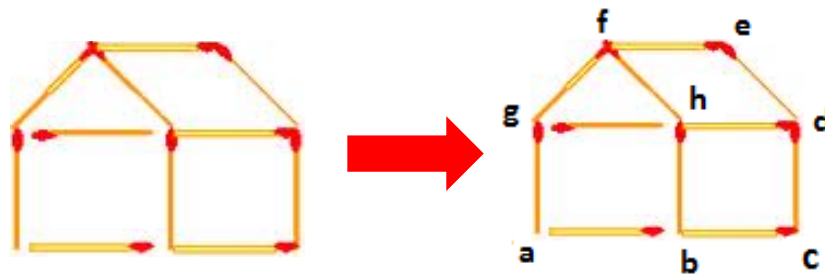
```
% dibuja( [d-b,c-a,d-a,c-b,a-b,d-c,d-e,e-c], L ).
```

```
SWI-Prolog (Multi-threaded, version 6.4.1)
File Edit Settings Run Debug Help
% library(win_menu) compiled into win_menu 0.00 sec, 34 clauses
Welcome to SWI-Prolog (Multi-threaded, 32 bits, Version 6.4.1)
Copyright (c) 1990-2013 University of Amsterdam, VU Amsterdam
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software,
and you are welcome to redistribute it under certain conditions.
Please visit http://www.swi-prolog.org for details.

For help, use ?- help(Topic). or ?- apropos(Word).

1 ?-
Warning: c:/syc/temp/usb/leng prgr/notas/programación lógica/programas/untrazo.pl:25
:
Singleton variables: [V2]
% c:/SYC/temp/USB/Leng Prgr/Notas/Programación Lógica/Programas/UnTrazo.pl compiled
0.00 sec, 6 clauses
1 ?- dibuja( [a-b,a-c,a-d,b-c,b-d,c-d,c-e,d-e], L ).
L = [a, b, c, a, d, c, e, d, b] ;
L = [a, b, c, a, d, e, c, d, b] ;
L = [a, b, c, d, a, c, e, d, b] ;
L = [a, b, c, d, e, c, a, d, b] ;
L = [a, b, c, e, d, a, c, d, b] ;
L = [a, b, c, e, d, c, a, d, b] ;
L = [a, b, d, a, c, d, e, c, b] ;
L = [a, b, d, a, c, e, d, c, b] ;
L = [a, b, d, c, a, d, e, c, b] ;
L = [a, b, d, c, e, d, a, c, b] ;
L = [a, b, d, e, c, a, d, c, b] ;
L = [a, b, d, e, c, d, a, c, b] ;
L = [b, a, c, b, d, c, e, d, a]
```

¿ Casa de cerillos de un sólo trazo ?



```
SWI-Prolog (AMD64, Multi-threaded, version 6.4.1)
File Edit Settings Run Debug Help
Copyright (c) 1990-2013 University of Amsterdam, VU Amsterdam
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software,
and you are welcome to redistribute it under certain conditions.
Please visit http://www.swi-prolog.org for details.

For help, use ?- help(Topic). or ?- apropos(Word).

1 ?-
Warning: c:/users/gustavo márquez/escuela/prolog/untrazo.pl:25:
Singleton variables: [V2]
% c:/Users/Gustavo Márquez/Escuela/Prolog/UnTrazo.pl compiled 0.00 sec, 6 clauses
1 ?- dibuja( [a-b,b-c,c-d,d-e,e-f,f-g,g-a,h-f,h-d,b-h],L).
false.
2 ?- █
```

El problema del recogedor



Solución al puzle.



%Un número es correcto y en su posición correcta

%El símbolo \+ representa la negación.

%member es una función que busca si el primer parámetro está contenido en el segundo

%Las comas representan la conjunción.

%El guión bajo representa una variable anónima que significa cualquier término

posicion_correcta(A, B, C, S) :- S = [A, _, _], \+ member(B, S), \+ member(C, S).

%Un número es correcto pero mal posicionado

mal_posicionado(A, B, C, S) :- (S = [_, A, _] ; S = [_, _, A]), \+ member(B, S), \+ member(C, S).

mal_posicionado(A, B, C, S) :- (S = [B, _, _] ; S = [_, _, B]), \+ member(A, S), \+ member(C, S).

%Dos números son correctos pero su posición no

%El \= representa la negación de ==

dos_mal_posicionados(A, B, C, S) :- S \= [A, _, _], \+ member(C, S).

dos_mal_posicionados(A, B, C, S) :- S \= [A, _, _], S \= [_, _, C], \+ member(B, S).

dos_mal_posicionados(A, B, C, S) :- S \= [_, B, _], \+ member(A, S).

%Nada es correcto

ninguno(A, B, C, S) :- \+ member(A, S), \+ member(B, S), \+ member(C, S).

solucion(S) :- between(0, 9, A), between(0, 9, B), between(0, 9, C).

%Las variables A, B y C toman valores en el rango del 0 al 9

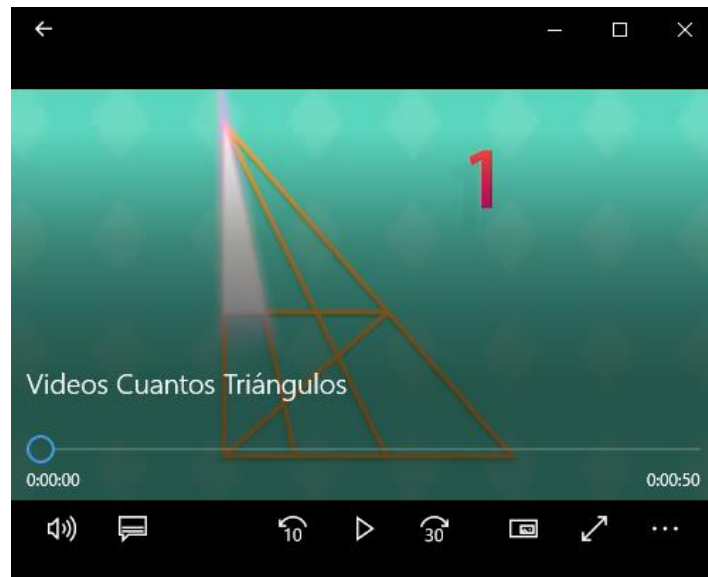
%S es la lista compuesta por las tres variables anteriores y en la que se evaluará las pistas.

S = [A, B, C], posicion_correcta(6, 8, 2, S), ninguno(7, 3, 8, S), mal_posicionado(7, 8, 0, S).

```
SWI-Prolog (AMD64, Multi-threaded, version 8.0.3)
File Edit Settings Run Debug Help
Welcome to SWI-Prolog (threaded, 64 bits, version 8.0.3)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit http://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

?-
% c:/Users/GustavoMarquez/Downloads/Candado.pl compiled 0.00 sec, 11 clauses
?- solucion( S ).
S = [0, 4, 2] ;
false.
?-
```



%puntos del polígono.

p(1).
p(7).
p(9).
p(11).

%línea de los triángulos.

linea([1,2,3,4]).
linea([1,5,6,10]).
linea([1,7,11]).
linea([3,6,9,11]).

%Verifica que dos puntos estén alineados en la lista L.

alineados(A,B) :- linea(L), member(A,L).

%Verifica que tres puntos estén alineados.

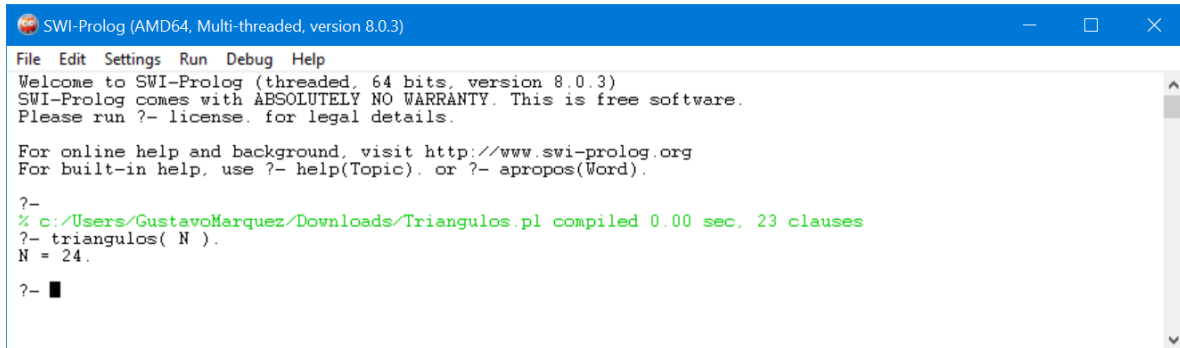
alineados(A,B,C) :- linea(L), member(A,L), member(B,L).

%verifica que los puntos dados formen un triángulo, para ello, tienen que estar alineados A con B, B con C y C
% con A, pero los tres no deben estar en la misma línea.

triangulo(A,B,C) :- alineados(A,B), alineados(C,A), (alineados(A,B,C)).

%findall es una función que encuentra todos los resultados, recibe tres parámetros, las variables, el predicado y la lista de elementos.(El predicado es un conjunto de reglas o hechos). La función length recibe dos parametros, el primero es la lista y el segundo es el una variable donde se guarda el tamaño de la lista.

triangulos(N) :- findall((A,B,C), triangulo(A,B,C), T).



```
SWI-Prolog (AMD64, Multi-threaded, version 8.0.3)
File Edit Settings Run Debug Help
Welcome to SWI-Prolog (threaded, 64 bits, version 8.0.3)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit http://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

?-
% c:/Users/GustavoMarquez/Downloads/Triangulos.pl compiled 0.00 sec, 23 clauses
?- triangulos( N ).
N = 24.
?-
```

✓ El paradigma de programación Funcional.

- LISP, ML, Miranda, Scheme.
- Haskell.

Problemas.

- Problema de las 8 reinas.

-- Problema de las ocho reinas de ajedrez en Haskell

{-# LANGUAGE NPlusKPatterns #-}

*-- El tablero es una Lista de números que indica las filas donde se deben
colocar las reinas:*

type Tablero = [Int]

-- Definición básica:

reinasAje :: Int -> [Tablero]

*reinasAje n = [r:rs | rs <- reinasAje(n - 1),
r <- ([1..4] \\ rs),
noAtaca r rs 1]*

noAtaca :: Int -> Tablero -> Int -> Bool

noAtaca r (a:rs) distH = abs(r - a) /= distH && noAtaca r rs (distH+1)

```
WinGHCi
File Edit Actions Tools Help
[Icons]

Prelude> :cd C:\SYC\temp\Leng Prgr\Notas\Programación Funcional\Programas Haskell
Prelude> :load "OchoReinas.hs"
[1 of 1] Compiling Main                ( OchoReinas.hs, interpreted )
Ok, modules loaded: Main.
*Main> reinasAje 4
[[3,1,4,2],[2,4,1,3]]
*Main> reinasAj 6
[[5,3,1,6,4,2],[4,1,5,2,6,3],[3,6,2,5,1,4],[2,4,6,1,3,5]]
*Main>
```

```
WinGHCi
File Edit Actions Tools Help
[Icons]

*Main> reinas 8
[[4,2,7,3,6,8,5,1],[5,2,4,7,3,8,6,1],[3,5,2,8,6,4,7,1],[3,6,4,2,8,5,7,1],[5,7,1,3,8,6,4,2],[4,6,8,3,1,7,5,2],[3,6,8,1,4,7,5,2],[5,3,8,4,7,1,6,2],[5,7,4,1,3,8,6,2],[4,1,5,8,6,3,7,2],[3,6,4,1,8,5,7,2],[4,7,5,3,1,6,8,2],[6,4,2,8,5,7,1,3],[6,4,7,1,8,2,5,3],[1,7,4,6,8,2,5,3],[6,8,2,4,1,7,5,3],[6,2,7,1,4,8,5,3],[4,7,1,8,5,2,6,3],[5,8,4,1,7,2,6,3],[4,8,1,5,7,2,6,3],[2,7,5,8,1,4,6,3],[1,7,5,8,2,4,6,3],[2,5,7,4,1,8,6,3],[4,2,7,5,1,8,6,3],[5,7,1,4,2,8,6,3],[6,4,1,5,8,2,7,3],[5,1,4,6,8,2,7,3],[5,2,6,1,7,4,8,3],[6,3,7,2,8,5,1,4],[2,7,3,6,8,5,1,4],[7,3,1,6,8,5,2,4],[5,1,8,6,3,7,2,4],[1,5,8,6,3,7,2,4],[3,6,8,1,5,7,2,4],[6,3,1,7,5,8,2,4],[7,5,3,1,6,8,2,4],[7,3,8,2,5,1,6,4],[5,3,1,7,2,8,6,4],[2,5,7,1,3,8,6,4],[3,6,2,5,8,1,7,4],[6,1,5,2,8,3,7,4],[8,3,1,6,2,5,7,4],[2,8,6,1,3,5,7,4],[5,7,2,6,3,1,8,4],[3,6,2,7,5,1,8,4],[6,2,7,1,3,5,8,4],[3,7,2,8,6,4,1,5],[6,3,7,2,4,8,1,5],[4,2,7,3,6,8,1,5],[7,1,3,8,6,4,2,5],[1,6,8,3,7,4,2,5],[3,8,4,7,1,6,2,5],[6,3,7,4,1,8,2,5],[7,4,2,8,6,1,3,5],[4,6,8,2,7,1,3,5],[2,6,1,7,4,8,3,5],[2,4,6,8,3,1,7,5],[3,6,8,2,4,1,7,5],[6,3,1,8,4,2,7,5],[8,4,1,3,6,2,7,5],[4,8,1,3,6,2,7,5],[2,6,8,3,1,4,7,5],[7,2,6,3,1,4,8,5],[3,6,2,7,1,4,8,5],[4,7,3,8,2,5,1,6],[4,8,5,3,1,7,2,6],[3,5,8,4,1,7,2,6],[4,2,8,5,7,1,3,6],[5,7,2,4,8,1,3,6],[7,4,2,5,8,1,3,6],[8,2,4,1,7,5,3,6],[7,2,4,1,8,5,3,6],[5,1,8,4,2,7,3,6],[4,1,5,8,2,7,3,6],[5,2,8,1,4,7,3,6],[3,7,2,8,5,1,4,6],[3,1,7,5,8,2,4,6],[8,2,5,3,1,7,4,6],[3,5,2,8,1,7,4,6],[3,5,7,1,4,2,8,6],[5,2,4,6,8,3,1,7],[6,3,5,8,1,4,2,7],[5,8,4,1,3,6,2,7],[4,2,5,8,6,1,3,7],[4,6,1,5,2,8,3,7],[6,3,1,8,5,2,4,7],[5,3,1,6,8,2,4,7],[4,2,8,6,1,3,5,7],[6,3,5,7,1,4,2,8],[6,4,7,1,3,5,2,8],[4,7,5,2,6,1,3,8],[5,7,2,6,3,1,4,8]]
*Main> |
```

Otros ejemplos.

-- Ordenación por inserción:

`ordena_por_insercion_1 :: Ord a => [a] -> [a]`

`ordena_por_insercion_1 ( x:xs ) = inserta x (ordena_por_insercion_1 xs)`

-- Ordenación por QuickSort:

ordenaQS :: Ord a => [a] -> [a]

ordenaQS (x:xs) = ordenaQS menores ++ [x] ++ ordenaQS mayores

where menores = [e | e <- xs, e < x]

mayores = [e | e <- xs, e >= x]